M2S-08: Step 8 — Expand: Adopt New SaaS Capabilities

> **Series:** M2S — Managed to SaaS Migration | **Notebook:** 8 of 9 | **Phase:** Run | **Step:** Expand | **Created:** March 2026 | **Last Updated:** 11/11/2026

With the migration complete and integrations reconnected, the real value of moving to SaaS begins. Dynatrace SaaS includes an entire generation of capabilities that were never available in Managed — Grail, Notebooks, OpenPipeline, Dynatrace Assist, AppEngine, and AutomationEngine. This notebook provides a structured approach to adopting each capability, with a recommended timeline that avoids overwhelming teams while ensuring steady progress.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run:** 7. Enable | **8. Expand** | 9. Optimize

Table of Contents

1. SaaS-Exclusive Capabilities Overview
2. Grail Adoption
3. Notebooks Adoption
4. OpenPipeline Configuration
5. Workflow Automation
6. Dynatrace Assist
7. Platform Apps
8. Privacy and Data Protection in SaaS

9. Feature Adoption Timeline
10. Step Completion Checklist

Prerequisites

Requirement	Details
Steps 1–6 Complete	Discovery, strategy, design, preparation, execution, and integration phases finished
SaaS Tenant Stable	OneAgents reporting, alerts flowing, integrations validated
Grail Enabled	Grail data lakehouse active and receiving data (default for SaaS)
Admin Access	Environment admin permissions for OpenPipeline, Workflows, and Settings
Team Readiness	Key stakeholders identified for each capability adoption

1. SaaS-Exclusive Capabilities Overview

These capabilities are available only in Dynatrace SaaS. They represent the primary reason to migrate beyond infrastructure parity.

Capability	What It Does	Priority
Grail	Unified data lakehouse — query logs, spans, events, metrics, and entities with DQL	High
Notebooks	Interactive analysis combining DQL queries, markdown documentation, and visualizations	High
Dynatrace Assist	AI-assisted natural language querying and root cause analysis	High
AppEngine	Build and deploy custom Dynatrace apps with the Dynatrace Developer toolkit	Medium

Capability	What It Does	Priority
AutomationEngine	Advanced workflow automation with event-driven triggers and multi-step actions	Medium
OpenPipeline	Real-time data processing, enrichment, routing, and masking at ingest	High
Grail Buckets	Custom data retention and routing by data type, environment, or compliance need	Medium
Segments	Reusable data scoping filters for dashboards and notebooks	Medium

> **Key Insight:** Do not try to adopt everything at once. A phased approach over 8–10 weeks prevents change fatigue and allows teams to build competency incrementally. The timeline in Section 9 provides a proven cadence.

2. Grail Adoption

Grail is the single most impactful new capability. It replaces USQL (User Sessions Query Language) with DQL (Dynatrace Query Language) and provides a unified query experience across all data types stored in the Grail data lakehouse.

What Changes from Managed

Managed Approach	SaaS with Grail
USQL for session data	DQL for all data types
Separate query interfaces per data type	Single DQL interface for logs, spans, events, metrics, entities
Limited cross-data-type analysis	Unified queries joining logs, spans, and entities
Retention set by the cluster's storage tiers	Configurable retention per Grail bucket
Metrics API for metric queries	`timeseries` command for metrics

Key Data Objects in Grail

Data Object	Description	Example
`logs`	Log records from all sources	`fetch logs, from:-1h`
`spans`	Distributed trace spans	`fetch spans, from:-1h`
`events`	Generic events (deployments, configuration changes)	`fetch events, from:-1h`
`bizevents`	Business events (transactions, conversions)	`fetch bizevents, from:-24h`
`dt.entity.*`	Entity data (hosts, services, processes)	`fetch dt.entity.host`
`dt.davis.problems`	Dynatrace Intelligence root cause problems	`fetch dt.davis.problems, from:-24h`
`dt.davis.events`	Dynatrace Intelligence raw events	`fetch dt.davis.events, from:-24h`

> **Important:** Metrics do NOT use `fetch`. Use the `timeseries` command: `timeseries avg(dt.host.cpu.usage), from:-1h`.

Your First DQL Queries

Start with queries that replace what you were doing in Managed. These demonstrate capabilities that were not possible before:

```
// Cross-data-type analysis: top services by error log volume
// Logs carry the service entity when OneAgent enriches them — no join needed
// status == "ERROR" also counts SEVERE/FATAL levels that loglevel == "ERROR" misses
fetch logs, from:-1h
| filter status == "ERROR" and isNotNull(dt.entity.service)
| summarize errorCount = count(), by:{dt.entity.service}
| sort errorCount desc
| limit 10
```

```
// Grail bucket usage — see how data is distributed across buckets
// Grail buckets are a SaaS-only concept for data routing and retention
fetch logs, from:-24h
| summarize logVolume = count(), by:{dt.system.bucket}
| sort logVolume desc
```

```
// Correlate detected problems with affected entities — unified in Grail
// smartscape.affected_entities replaces the deprecated affected_entity_ids. The classic field still
// works, but it names only ENVIRONMENT-0000000000000001 for entities that exist only in Smartscape,
// so that ID tops a classic ranking (3,370 of 7 days' closed problems on the validation tenant,
// 11/11/2026). Problems with no Smartscape node at all (environment-level alerts) drop out here.
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| expand entity = smartscape.affected_entities
| filter isNotNull(entity)
| summarize problemCount = count(), by:{entity_id = entity[id], entity_type = entity[type], entity_name = entity[name]}
| sort problemCount desc
| limit 10
```

```
// Metrics query using timeseries — replaces Metrics API queries in Managed
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| filter avgCpuValue &gt; 0
```

USQL to DQL Migration Patterns

For teams accustomed to USQL, here are common translation patterns. RUM data lives in `user.sessions` and `user.events` on Grail — not in `bizevents` or `logs`:

USQL Pattern	DQL Equivalent
`SELECT COUNT(*) FROM usersession`	`fetch user.sessions, from:-1h | summarize c = count()`
`FROM useraction`	`fetch user.events, from:-1h` + a filter on the action type (field names differ between classic RUM on Grail and New RUM)

USQL Pattern	DQL Equivalent
`WHERE country = 'US'`	`| filter country == "..."` (classic) or `| filter geo.country.iso_code == "US"` (New RUM)
`ORDER BY count(*) DESC`	`| summarize c = count(), by:{...} | sort c desc` — alias the aggregation first
`LIMIT 10`	`| limit 10`
`BETWEEN "2024-01-01" AND "2024-01-31"`	`fetch ..., from:"2024-01-01T00:00:00Z", to:"2024-01-31T00:00:00Z"`

> **Tip:** DQL uses a pipeline model (`|`) rather than SQL clauses. **WEBRUM-09** has the full grammar and field mapping for both RUM generations, and lists the USQL fields (such as `city`) with no New RUM equivalent.

3. Notebooks Adoption

Dynatrace Notebooks replace ad-hoc querying with persistent, shareable, collaborative analysis documents. They combine executable DQL queries with markdown documentation and visualizations in a single workspace.

Why Notebooks Matter for Migration Teams

Use Case	How Notebooks Help
Migration validation	Document pre/post migration metrics side by side
Incident investigation	Build reusable investigation runbooks with live DQL
Capacity planning	Combine metrics timeseries with written analysis
Executive reporting	Create self-updating reports with embedded queries
Team onboarding	Write training notebooks that new team members can execute
Knowledge sharing	Share analysis methodology alongside the actual queries

Recommended Starter Notebooks

Create these notebooks in the first two weeks to build team familiarity:

Notebook	Purpose	Audience
Migration Health Check	Compare key metrics pre- and post-migration	Platform team
Error Log Analysis	Top errors by service with trend analysis	SRE team
Host Inventory	All monitored hosts with agent versions and status	Operations
Problem Summary	Weekly problem trends with MTTR analysis	Management
DQL Cheat Sheet	Common query patterns for the team to reference	All users

Creating Your First Notebook

1. Navigate to **Notebooks** in the SaaS left navigation
2. Click **Create notebook**
3. Add a **Markdown section** with a title and description
4. Add a **DQL section** with a query
5. Execute the query to see results inline
6. Share the notebook with your team

> **Tip:** Notebooks support version history. You can revert to any previous version, making them safe for experimentation.

Verify Notebook Availability

Confirm that Notebooks are accessible and DQL queries execute correctly:

```
// Simple validation query — if this returns data, Grail and Notebooks are
working
fetch logs, from:-15m
| summarize logCount = count()
| fieldsAdd status = if(logCount &gt; 0, then: "Grail is receiving logs",
else: "No logs in last 15 minutes — check OneAgent connectivity")
```

4. OpenPipeline Configuration

OpenPipeline is the SaaS-exclusive data processing engine that operates at ingest time. It goes well beyond the log processing available in Managed, with a full pipeline that can enrich, transform, route, and mask data before it reaches Grail.

What OpenPipeline Provides

Capability	Description	Managed Equivalent
Log enrichment	Add fields, parse structured data at ingest	Limited custom log attributes
Data routing	Route logs to specific Grail buckets by rules	Not available
PII masking	Redact sensitive data at ingest	OneAgent-side masking rules (still available on SaaS)
Data transformation	Normalize, rename, and restructure fields	Not available
Metric extraction	Generate metrics from log data	Log metrics (limited)
Drop rules	Discard noisy or irrelevant data before storage	Not available

Configuration Steps

1. **Navigate to OpenPipeline** — Settings > OpenPipeline (or search in the SaaS UI)
2. **Select data type** — Logs, Events, Spans, or Metrics
3. **Add processing rules** — Define enrichment, masking, and routing rules
4. **Configure Grail buckets** — Create custom buckets with appropriate retention

5. **Set routing rules** — Direct data to the correct bucket based on conditions
6. **Test with sample data** — Verify rules process correctly before applying broadly

Grail Bucket Strategy

Create buckets based on data lifecycle and compliance requirements:

Bucket Name	Contents	Retention	Rationale
`default_logs`	General application logs	35 days	Standard operational use
`security_logs`	Audit and authentication logs	365 days	Compliance requirement
`debug_logs`	Debug-level logs	7 days	High volume, low long-term value
`infrastructure_logs`	Host and container logs	14 days	Troubleshooting window

Verify OpenPipeline Is Processing Data

```
// Check data distribution across Grail buckets
// If you see only "default_logs", OpenPipeline routing rules have not been configured yet
fetch logs, from:-1h
| summarize logCount = count(), by:{dt.system.bucket}
| sort logCount desc
```

```
// Check log sources flowing into Grail — verify all expected sources are present
fetch logs, from:-1h
| summarize logCount = count(), by:{log.source}
| sort logCount desc
| limit 20
```

PII Masking Configuration

OpenPipeline masks sensitive data at ingest, before storage. Two things to know before you write the rules:

- **The pattern language is DPL, not regex.** OpenPipeline matchers use the Dynatrace Pattern Language, while OneAgent-side masking uses RE2 regex — so the same masking job uses two languages depending on where it runs. A regex such as `\d{3}-\d{2}-\d{4}` pasted into an OpenPipeline rule will not behave as it does in a regex engine. **FAQ-15** covers the translation and the traps (DPL performs no backtracking), and **OPLOGS-08** covers masking in the pipeline.
- **Mask as early as the data allows.** OneAgent-side masking keeps the value from ever leaving the host; OpenPipeline masking covers every ingest channel, including API and OpenTelemetry sources that OneAgent never sees.

Data type	What to mask
National ID numbers (e.g. SSNs)	The full value
Payment card numbers	All but the last four digits
Email addresses	The local part, or the whole value
IP addresses (if required)	The last octet

Masking rules apply at ingest — once data enters Grail, it is already masked. Records ingested before a rule existed are not rewritten.

5. Workflow Automation

The AutomationEngine replaces Managed problem notifications with event-driven workflows that can execute multi-step actions, call external APIs, and implement automated remediation.

What Replaces What

Managed Feature	SaaS Replacement
Problem notification rules	Workflow triggers on detected problems
Custom webhooks	HTTP Request action in Workflows
Maintenance windows	Workflow-aware maintenance windows
Manual escalation	Automated escalation with conditional logic
Scheduled reports (manual)	Scheduled Workflow execution with report generation

Workflow Building Blocks

Component	Purpose	Example
Trigger	What starts the workflow	detected problem opened, schedule (cron), event ingest
Action	What the workflow does	Send Slack message, call API, run DQL, create Jira ticket
Condition	Control flow between actions	If severity is CRITICAL, escalate; otherwise, log
Loop	Repeat actions	Retry an API call up to 3 times

Recommended Starter Workflows

Workflow	Trigger	Actions
Problem alerting	detected problem opened	Run DQL for context > Send to Slack with enriched details
Daily health report	Scheduled (daily at 8 AM)	Run DQL queries > Generate summary > Send email
Auto-remediation	Specific detected event type	Verify condition > Execute remediation script > Validate fix
Deployment tracking	Custom deployment event	Record deployment > Compare error rates before/after

> **Tip:** Start by converting your most critical Managed notification rules to Workflows. Test each workflow end-to-end before retiring the Managed equivalent.

Verify Workflow Execution

After creating workflows, verify they are triggering correctly:

```
// Check recent detected problems — these should be triggering your workflows
fetch dt.davis.problems, from:-24h
| fieldsKeep display_id, event.name, event.status, event.category, timestamp
| sort timestamp desc
| limit 10
```

```
// Problem trends over 7 days — baseline for workflow trigger validation
fetch dt.davis.problems, from:-7d
| makeTimeseries problemCount = count(default: 0), interval: 24h
```

6. Dynatrace Assist

Dynatrace Assist brings AI-assisted querying and analysis to Dynatrace. Users can ask questions in natural language and Copilot generates DQL queries, explains results, and assists with root cause analysis.

What Dynatrace Assist Can Do

Capability	Description
Natural language queries	Ask "Show me error logs from the last hour" and get DQL
Query explanation	Paste a DQL query and get a plain-English explanation
Root cause analysis	Ask about a specific problem and get AI-driven investigation
Notebook integration	Use Copilot directly within Notebooks for guided analysis
Dashboard interpretation	Ask Copilot to explain what a dashboard is showing

Enabling Dynatrace Assist

1. Confirm generative AI is enabled for the environment — the setting and the IAM permissions that govern who may use it are described in the Dynatrace Assist documentation linked below (AIOPS-04 covers the controls)
2. Grant the permission to the groups that should use it
3. Assist then appears in Notebooks, Dashboards, and the Dynatrace search

Adoption Strategy

Dynatrace Assist is most valuable for:

- **New DQL users** — Copilot generates queries from natural language, accelerating the learning curve
- **Incident responders** — Natural language investigation during high-pressure incidents
- **Executives** — Ask business questions without needing DQL knowledge

> **Recommended adoption:** Weeks 7–8 post-migration. By this point teams have basic DQL literacy, and Copilot becomes a force multiplier rather than a crutch.

7. Platform Apps

The Dynatrace Hub provides a marketplace of pre-built apps, and AppEngine allows you to build custom apps tailored to your organization.

Exploring the Dynatrace Hub

Navigate to **Hub** in the SaaS left navigation to browse available apps:

App Category	Examples	Value
SLO management	SLO apps for tracking and reporting	Formalize reliability objectives
Release comparison	Compare deployments side by side	Deployment confidence
Security	Vulnerability analytics, code-level security	Shift-left security
Cloud optimization	Cloud cost and resource recommendations	FinOps visibility
Ownership	Service ownership and team management	RACI clarity

Building Custom Apps with AppEngine

For organization-specific needs, AppEngine provides:

- **TypeScript/React framework** — Build with familiar web technologies
- **Dynatrace Design System** — Consistent UI components (Strato)
- **DQL data access** — Query Grail directly from your app
- **Platform API access** — Full Dynatrace API integration
- **Built-in authentication** — Users authenticate with their Dynatrace account

When to Build Custom Apps

Build If	Use Hub App If
You need organization-specific logic or branding	A Hub app already solves the use case
You need to integrate proprietary data sources	Standard Dynatrace data is sufficient
You need custom user experiences for specific teams	Standard dashboards and notebooks meet the need
You need to embed Dynatrace data in external portals	Users can access Dynatrace directly

> **Tip:** Start with Hub apps before investing in custom development. Many common use cases already have well-maintained apps in the Hub.

Verify Platform App Availability

```
// Verify your environment is receiving data across all key data types
// Platform apps depend on Grail data being available
fetch logs, from:-1h
| summarize logCount = count()
| append [
    fetch spans, from:-1h
    | summarize spanCount = count()
  ]
| append [
    fetch events, from:-1h
    | summarize eventCount = count()
  ]
```

8. Privacy and Data Protection in SaaS

Moving to SaaS introduces new data protection capabilities that were not available in Managed. These are particularly important for organizations subject to GDPR, HIPAA, PCI-DSS, or similar regulations.

Data Protection Capabilities

Capability	Where to Configure	Purpose
PII masking via OpenPipeline	Settings > OpenPipeline	Redact sensitive data at ingest before storage
Session replay masking	Application settings > Session Replay	Mask form fields, inputs, and sensitive page elements
IP anonymization	Application settings > Data privacy	Anonymize end-user IP addresses
URL parameter exclusion	Application settings > Data privacy	Strip sensitive query parameters from captured URLs
Log redaction rules	Settings > OpenPipeline > Logs	Apply regex-based masking to log content
Do Not Track	Application settings > Data privacy	Respect browser DNT headers

GDPR Considerations

Topic	Action
Data Processing Addendum (DPA)	Ensure your DPA with Dynatrace covers SaaS processing
Data residency	Verify your SaaS cluster is in the appropriate geographic region
Data retention	Configure Grail bucket retention to match your data minimization requirements
Right to erasure	Understand how to request data deletion from Grail
Data access logging	Use audit logs to track who accesses what data

Session Replay Privacy

If you use Session Replay (Digital Experience Monitoring), configure masking levels:

Masking Level	What It Masks	When to Use
Mask user input	All text typed by users	Default — recommended minimum
Mask all text	All visible text on the page	High-privacy environments
Block specific elements	Targeted CSS selectors	Mask specific sensitive areas
Allow list	Only capture explicitly allowed elements	Most restrictive — compliance-driven

9. Feature Adoption Timeline

This timeline begins after Step 6 (Integrate) is complete and the migration is stable. Adjust the pace based on your team's capacity and organizational change tolerance.

Week	Focus Area	SaaS Capability	Key Activities
1–2	Stabilize	Core monitoring	Tune alerts, validate data parity, fix integration issues
3–4	Query and Analyze	Notebooks, Grail, DQL	Create team notebooks, train on DQL basics, build starter queries
5–6	Process and Protect	OpenPipeline, Grail Buckets	Configure log processing, create custom buckets, set up PII masking
7–8	Automate	AutomationEngine, Dynatrace Assist	Convert notification rules to Workflows, enable Copilot
9–10	Extend	Platform Apps, AppEngine	Explore Hub apps, evaluate custom app needs, deploy first apps

Success Metrics for Each Phase

Phase	Metric	Target
Weeks 1–2	Alert false positive rate	Below Managed baseline
Weeks 3–4	Notebooks created by team	At least 5 active notebooks
Weeks 5–6	Data routed to custom buckets	At least 2 custom buckets active
Weeks 7–8	Workflows replacing notifications	At least 3 active workflows
Weeks 9–10	Hub apps deployed	At least 2 apps installed

10. Step Completion Checklist

Do not proceed to Step 9 (Optimize) until the high-priority items are confirmed.

Checkpoint	Priority	Status
Grail querying adopted (DQL replacing USQL)	High	[ ]
Team notebooks created for key use cases	High	[ ]
OpenPipeline configured for log processing	High	[ ]
Custom Grail buckets created with retention policies	Medium	[ ]

Checkpoint	Priority	Status
PII masking rules configured in OpenPipeline	High	[ ]
Session replay masking configured (if DEM is active)	Medium	[ ]
First workflow automation implemented	Medium	[ ]
Dynatrace Assist enabled and accessible to teams	High	[ ]
Platform apps explored in Hub	Medium	[ ]
Feature adoption timeline shared with stakeholders	High	[ ]
Data residency and GDPR requirements verified	High	[ ]

Next Step

> **M2S-09: Step 9 — Optimize** — Validate the migration, optimize SaaS configuration, decommission the Managed environment, and establish ongoing operational excellence.

Additional Resources

- Grail Data Lakehouse Documentation
- DQL Documentation
- Notebooks Documentation
- OpenPipeline Documentation
- AutomationEngine (Workflows) Documentation
- Dynatrace Assist Documentation
- AppEngine Documentation
- Dynatrace Hub

Summary

In Step 8, you:

- Surveyed the full set of SaaS-exclusive capabilities and prioritized adoption
- Adopted Grail as the unified query platform, replacing USQL with DQL
- Created team Notebooks for collaboration, investigation, and reporting
- Configured OpenPipeline for log processing, data routing, and PII masking
- Implemented workflow automation to replace Managed notification rules
- Enabled Dynatrace Assist for AI-assisted querying and root cause analysis
- Explored platform apps in the Dynatrace Hub
- Addressed privacy and data protection requirements specific to SaaS
- Established a phased feature adoption timeline for the broader organization

> **Key Takeaway:** The Expand step is where the migration pays dividends. Every capability adopted here delivers value that was impossible on Managed. Prioritize Grail, Notebooks, and OpenPipeline first — they have the highest immediate impact.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.